

HPC- Merge sort

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

void takeUserInputArray(vector<int>& arr, int size) {
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void parallelMergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, left, mid);
            #pragma omp section
            parallelMergeSort(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

    vector<int> arr(n);
    takeUserInputArray(arr, n);

    auto start = high_resolution_clock::now();
    parallelMergeSort(arr, 0, n - 1);
    auto stop = high_resolution_clock::now();
}
```

```

    auto parMergeTime = duration_cast<milliseconds>(stop -
start).count();

    // Displaying the sorted array
    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Displaying the execution time for parallel merge sort
    cout << "Parallel Merge Sort Execution Time: " << parMergeTime << "
ms\n";

    return 0;
}

```

```

o/p:
Enter the size of the array: 5
Enter 5 elements:
5 9 2 1 3
Sorted Array: 1 2 3 5 9
Parallel Merge Sort Execution Time: 0 ms

```